Delegation by proposal and verification: a research plan

19 October 2026

1 Answer and position

The author asks: "The algorithmic approach seems to not be fruitful. Can this instead be formulated as an agentic framework where delegates are computed by agents s.t. algebraic constraints derived from the contract calculus? If so, can it be done using strand labs?" **Yes, under three conditions.** (1) Agents only propose. A family of contracts is accepted only when a deterministic verifier (Python and Z3) proves the conditions of the contract calculus for it, so correctness never rests on an agent. (2) What needs the whole graph stays symbolic: the set N of relevant components, forwarding where it applies, and refutation by the existence sentences Φ and Φ° of `complete.tex` (all defined in Section 2). (3) The author accepts three losses: the result is not guaranteed to be maximal; outside nested orderings only a budget guarantees that the search stops; the result is canonical only if the proposer is a deterministic enumerator. **Strands: yes, for the proposers only.** The Strands Agents SDK can run one agent per component on the Anthropic API, return typed proposals and log every model call. Strands Labs offers AI Functions, an experimental decorator that re-asks the model, up to a fixed number of attempts, until a Python check passes; it fits the per-component checks. Neither provides the verifier or coordinator (deterministic Python); a plain SDK loop is about as short.

What changes. The calculus of `docs/complete.tex` still fixes what a correct delegation is: (V1), (V2), (R), (K) below. What moves is the search. There, it decides Φ by quantifier elimination over all timed variables, builds the seed, and sweeps; each sweep step eliminates every timed variable the component cannot see from a formula holding all other current sets. Here a proposer writes a contract and quantifier-free validity queries check it (or, for (K) without a rule list, one sentence over the component's ports). Maximality is given up, because deciding it is an $\exists\forall$ question that only the sweep answers. Selection itself is not new: by the proposition "no greatest solution" of `complete.tex`, no acceptable delegation contains all others, so `complete.tex` already selected one by fixed orders; now proposers do.

2 The constraints on a proposed family

Notation, as in `complete.tex`. V is a finite set of real-valued variables. Component i reads $\mathrm{in}(i)$ and writes $\mathrm{out}(i)$; $\mathrm{ports}(i) := \mathrm{in}(i) \cup \mathrm{out}(i)$. A variable written by no component is an *environment variable*. Time is $t \in \{0, \dots, H\}$; the *timed variable* $v(t)$ is the value of v at time t. A *history* of a set W of variables gives a value to every $v(t)$ with $v \in W$; $\overline{W}$ is the set of timed variables $v(t)$ with $v \notin W$. The language $\mathcal{L}$ consists of Boolean combinations of atoms $\sum_v a_v\, v(t) \le b$, rational a_v, b, one time per atom. A bounded PolySTL formula ψ is compiled into $\mathcal{L}$ by $[\psi]_0$; $\Box_{[a,b]}$ and $\Diamond_{[a,b]}$ mean "at every" and "at some" time in $[t+a, t+b]$. $\varphi \models \psi$ means every trace (assignment of reals to all timed variables) satisfying φ satisfies ψ. A contract is a pair (A, G) of formulas, assumption and guarantee. Its implementation set is $S := \neg A \vee G$; the contract is *saturated*, written (A, S). For a contract Q, A_Q and S_Q are its assumption and implementation set. The query is $P = (A_P, G_P)$, and $S_P := \neg A_P \vee G_P$. Refinement: $(A, S) \preccurlyeq (A', S')$ iff $A' \models A$ and $S \models S'$. Projection onto W (what can be said over W) is $\rho_W(A, S) := (\exists\overline{W} A, \forall\overline{W} S)$. Composition (side by side) and quotient (what a component must meet so that, next to one meeting (A', S'), the pair meets (A, S)):

$$(A_1, S_1) \otimes (A_2, S_2) := \big((A_1 \wedge A_2) \vee \neg(S_1 \wedge S_2),\, S_1 \wedge S_2\big), \qquad (A, S)/(A', S') := \big(A \wedge S',\, (S \wedge A') \vee \neg(A \wedge S')\big).$$

N is the set of components that write a variable of G_P, together with every component from which a directed path leads to one of them. (Scope): every variable of A_P or G_P is a port of a component of N. For component i and given sets S_j: $X_i := \overline{\mathrm{ports}(i)}$ (what i cannot see), and $R_i := A_P \wedge \bigwedge_{j \ne i} S_j$ (what the rest allows). $\mathrm{in}_i(t)$ and $\mathrm{out}_i(t)$ are the timed variables at time t of $\mathrm{in}(i)$ and $\mathrm{out}(i)$. A *causal strategy* for i gives, for each $t \le H$, the values of $\mathrm{out}_i(t)$ as a function of the values of $\mathrm{in}(i)$ at times $\le t$. A contract $C_i = (A_i, S_i)$ of i is *receptive* (R) if for every history of $\mathrm{in}(i)$ some history of $\mathrm{out}(i)$ gives a trace in S_i, and *causal* (K) if some causal strategy produces, from every history of $\mathrm{in}(i)$, a trace in S_i; (K) implies (R). A family (C_i), one per component, is an *acceptable delegation* if each C_i is over $\mathrm{ports}(i)$ and (V1), (V2) of the table below, (R) and (K) hold. A topological ordering of N (each component after those whose outputs it reads) is *nested* if each component feeds the next and can copy onto that edge all the next cannot see; *forwarding* is that copying (`complete.tex`, carriers lemma). Φ is the sentence of step 3 of `complete.tex` (for every input there is an output, component by component along a nested ordering); it decides existence there, and the *seed* is the delegation it yields (step 7). Φ° is its variant for any topological ordering: false proves that none exists. The *sweep* is steps 8–9, quotient and projection.

Causal rule lists. A *causal rule list* f_i for component i gives, for each $y \in \mathrm{out}(i)$ and $t \le H$, a rule $f_{y,t}$: "if π_1 then c_1, else if π_2 then c_2, …, else 0". Each region π_l is a formula of $\mathcal{L}$ over $\mathrm{in}(i)$ at times $\le t$. Each value c_l is affine, with rational coefficients, in $\mathrm{in}_i(t)$. $S_i[f_i]$ is S_i with every $y(t)$ replaced by $f_{y,t}$, written in $\mathcal{L}$: an atom α becomes $\bigvee_l (\pi_l \wedge \bigwedge_{l'<l} \neg\pi_{l'} \wedge \alpha[y(t) := c_l]) \vee (\bigwedge_l \neg\pi_l \wedge \alpha[y(t) := 0])$, applied for each $y(t)$ in turn. The *defining formula* is $[\![f_i]\!] := \bigwedge_{y,t} [\![y(t) = f_{y,t}]\!]$, with $[\![y(t) = f_{y,t}]\!] := \bigvee_l (\pi_l \wedge \bigwedge_{l'<l} \neg\pi_{l'} \wedge y(t) = c_l) \vee (\bigwedge_l \neg\pi_l \wedge y(t) = 0)$.

The constraints. A proposal gives, for each $i \in N$, formulas A_i, G_i of $\mathcal{L}$ (or bounded PolySTL, compiled), and optionally a causal rule list f_i. Put $S_i := \neg A_i \vee G_i$; saturation then holds by definition and needs no check. The family is *accepted* iff all rows hold:

constraint	statement	Z3 check	quantifiers, variables
(C0) scope	$(A_i, G_i) = (\mathsf{true}, \mathsf{true})$ for $i \notin N$	none: set by the coordinator	none
(C1) ports	only $v(t)$ with $v \in \mathrm{ports}(i)$, $t \le H$	parse, only these declared	none: syntax
(V1)	$A_P \wedge \bigwedge_i S_i \models G_P$	$A_P \wedge \bigwedge_i S_i \wedge \neg G_P$ unsat	none; all $\lvert V\rvert(H+1)$
(V2), each j	$A_P \wedge \bigwedge_i S_i \models A_j$	$A_P \wedge \bigwedge_i S_i \wedge \neg A_j$ unsat	none; all $\lvert V\rvert(H+1)$
(K), rule list	f_i causal, $S_i[f_i]$ valid	syntax, then $\neg S_i[f_i]$ unsat	none; $\lvert\mathrm{in}(i)\rvert(H+1)$
(K), no list	$\forall \mathrm{in}_i(0) \exists \mathrm{out}_i(0) \cdots \forall \mathrm{in}_i(H) \exists \mathrm{out}_i(H)\, S_i$	Z3 on this sentence	$2(H+1)$ blocks; $\lvert\mathrm{ports}(i)\rvert(H+1)$

By the validity proposition of `complete.tex`, (C1), (V1), (V2), (K) say exactly "acceptable delegation". (C0) loses nothing: outputs of components outside N occur in no formula of N, A_P, G_P, and (R) fills them in.

Lemma 1 (rule lists lose nothing)**.** *(a) If f_i is causal and $S_i[f_i]$ is valid, then S_i satisfies (K). (b) If $S_i \in \mathcal{L}$ satisfies (K), a causal rule list f_i with rational data and $S_i[f_i]$ valid exists.*

Proof. (a) The rule list is a causal strategy; validity says its traces lie in S_i. (b) Eliminate the quantifiers of the (K) sentence from the inside out, recording witnesses (elimination with witnesses, `complete.tex`, parts (b), (c)): values are affine at their time with rational coefficients and regions are in $\mathcal{L}$. The witness of $y(t)$ mentions only inputs of i at times $\le t$, outputs of i at times $< t$ (in regions only), and outputs of i at time t earlier in the block (in regions and values). Substitute the witnesses of these outputs, earliest first: in a region, replace each atom α mentioning z by $\bigvee_{(\pi', c')} (\pi' \wedge \alpha[z := c'])$ as in step 6; in a value c mentioning $z(t)$, replace the pair (π, c) by the pairs $(\pi \wedge \pi', c[z(t) := c'])$. Values stay affine at time t, regions stay in $\mathcal{L}$, and each round brings in only earlier variables, so this ends. The induction of the (V1) part of the soundness theorem, on this one sentence, puts the traces in S_i. □

Lemma 2 (the quotient is checked, never computed)**.** *Fix all contracts but C_i, and let $Q_i := \rho_{\mathrm{ports}(i)}\big(P / \bigotimes_{j \neq i} C_j\big)$, the projected quotient. (a) $S_{Q_i} \equiv \forall X_i \big(R_i \to G_P \wedge \bigwedge_{j \neq i} A_j\big)$ and $A_{Q_i} \equiv \exists X_i R_i$. (b) $C_i \preccurlyeq Q_i$ iff the two quantifier-free validities $R_i \wedge S_i \models G_P \wedge \bigwedge_{j \neq i} A_j$ and $R_i \models A_i$ hold. (c) A rational history z of $\mathrm{ports}(i)$ lies in S_{Q_i} iff $R_i \wedge (\mathrm{ports}(i) = z) \wedge \neg(G_P \wedge \bigwedge_{j \neq i} A_j)$ is unsatisfiable, where $(\mathrm{ports}(i) = z)$ fixes every timed port to its value in z: one quantifier-free call.*

Proof. (a) The others compose to (A', S'), $S' = \bigwedge_{j \neq i} S_j$, $A' = \bigwedge_{j \neq i} A_j \vee \neg S'$. The quotient $P/(A', S') = \big(A_P \wedge S',\ (S_P \wedge A') \vee \neg(A_P \wedge S')\big)$ has assumption R_i, and, as $S_P = G_P$ and $A' = \bigwedge_{j \neq i} A_j$ on R_i, guarantee $R_i \to G_P \wedge \bigwedge_{j \neq i} A_j$; $\rho_{\mathrm{ports}(i)}$ takes $\exists X_i$ of the first and $\forall X_i$ of the second. (b) Refinement against (a); S_i and A_i do not mention X_i, so $S_i \models \forall X_i \varphi$ iff $S_i \models \varphi$, and $\exists X_i R_i \models A_i$ iff $R_i \models A_i$. (c) By (a). □

As $R_i \wedge S_i = A_P \wedge \bigwedge_j S_j$ and $\neg A_i \models S_i$, (b) is (V1), (V2) seen from i. Only the sweep, which computes S_{Q_i}, eliminates what i cannot see.

3 The loop

Roles. A *proposer* p_i for each $i \in N$ sees the graph, the query P, its port list $\mathrm{ports}(i)$, the current family, and the counterexamples projected onto $\mathrm{ports}(i)$. It outputs a contract over $\mathrm{ports}(i)$ in bounded PolySTL or $\mathcal{L}$, optionally with a causal rule list, or declines; it is outside the trust base. The *verifier* (deterministic Python with Z3, fixed version and seed) checks (C1), (K), (V2), (V1) in this order and stops at the first failure with a counterexample. (V2) comes first: a (V1) counterexample that violates some A_j satisfies $A_P \wedge \bigwedge_i S_i \wedge \neg A_j$, so it is a (V2) counterexample, and the right fault to report, since off A_j saturation lets j do anything. Once (V2) holds, every (V1) counterexample satisfies all A_j. The *coordinator* (deterministic Python) holds the routing policy, cell projection, the no-repeat record and budget B.

No culprit. If a trace x violates (V1), then for every i the restriction of x to $\mathrm{ports}(i)$ lies in S_i but not in S_{Q_i} ($x \models R_i \wedge S_i \wedge \neg G_P$, then Lemma 2(c)). So the algebra does not say which contract must change; routing is a policy.

Cells, not points. A proposer can exclude a counterexample point while every nearby trace still fails, so repair against single points need not converge. For a counterexample x of an obligation ω (for (V1), $\omega = A_P \wedge \bigwedge_i S_i \wedge \neg G_P$), the *cell* of x is the conjunction, over the atoms α of ω, of α if $x \models \alpha$ and $\neg\alpha$ otherwise. Every trace in the cell is a counterexample. The *projected cell* for i is $\exists X_i\, \mathrm{cell}(x)$: elimination from one conjunction (Fourier–Motzkin, no case split). If ω is the (V1) obligation, or the (V2) obligation of some $j \neq i$, every history in the projected cell lies outside S_{Q_i} (Lemma 2(c)); for (V1) it stays outside while the other sets only grow, for (V2) only while A_j is also unchanged. For the (V2) obligation of i itself, the projected cell lies inside A_{Q_i} and violates A_i: A_i is too strong. Proposer i keeps its cells in a set $\mathit{forbidden}_i$, emptied whenever another set shrinks, since old cells may then be allowed. Dually, a history $z \notin S_i$ with $z \in S_{Q_i}$ is a *positive example*: S_i may be enlarged to include it.

Routing. (1) On a (V2) failure for j: ask the writers other than j of the non-environment variables of A_j, in topological order, to exclude their projected cells; if there are none, or all decline, ask j to weaken A_j. (2) On a (V1) failure: ask in reverse topological order, so the writers of variables of G_P first. The one asked may strengthen G_i to exclude its projected cell while keeping (K); add an assumption over $\mathrm{in}(i)$ that is false on the cell, which moves the obligation to its writers (case 1 next round); or decline, and the next is asked. (3) On a (K) failure for i: return the failing input history if a rule list was given; if (K) was checked by the sentence, the verifier returns no history and i is told only "(K) failed". (4) A rejected triple (component, proposal, hash of the others) proposed again counts as declining.

Repair mode and weakening mode. *Repair* starts from a family that fails and routes counterexamples as above. *Weakening* starts from an accepted family. A proposal S_i' replaces S_i if it passes (C1), $S_i \models S_i'$, and (V1), (V2) hold for the new family: a syntax check and three quantifier-free checks. (R) and (K) are inherited, because they survive enlarging S_i. Every state is acceptable, so the loop can stop at any time. Proposers receive positive examples, each found by one call of Lemma 2(c). The loop, with input the graph, P, H, the budget B and the proposers (Φ counts as cheap when $|V|(H+1) \le 12$, the governor's size at $H = 2$; line 7 re-checks the seed, acceptable by `complete.tex`):

```
 1  compute N; if (Scope) fails: return "does not apply"; C_i := (true,true), i not in N
 2  if a nested ordering exists and |V|(H+1) <= 12:
 3      if Phi is false: return "none exists"
 4      F := seed of complete.tex;  mode := WEAKEN
 5  elif Phi-circle is false for some topological ordering: return "none exists"
 6  else: F := { p_i.propose(graph, P, ports(i)) : i in N };  mode := REPAIR
 7  best := F if verify(F) = ACCEPT else NONE;  C := none;  declined := {}
 8  for round := 1 .. B:
 9      if mode = REPAIR:
10          r := verify(F)                      # (C1), (K), (V2), (V1); first failure
11          if r = ACCEPT: best := F; mode := WEAKEN; continue
12          if C is not none: rejected += {(i, C, hash of the others)}
13          i := route(r, F, declined)          # next one to ask on this failure
14          if i is none: break                 # all asked components declined
15          C := p_i.propose(..., projected cell or (K) feedback, forbidden_i)
16          if C = DECLINE or (i, C, hash) in rejected: declined += {i}; C := none
17          else: F := F with C_i := C;  declined := {}
18      else:                                   # WEAKEN
19          i := next component of N in a fixed cyclic order
20          Z := sampled histories z of ports(i) with z not in S_i and z in S_Q_i
21          if Z is empty for every i in one full cycle: break
22          C := p_i.propose(..., allowed := Z)
23          if C passes (C1), S_i |= S_C, (V1), (V2): F := F with C_i := C;  best := F
24      log(round, i, raw proposal, parsed formula, verdict, counterexample)
25  return best with its rule lists and the log, or "not found within budget"
```

Where the algorithmic pieces remain. N and (Scope): always. Φ: when cheap (0.03 s on the governor at $H = 2$). The seed: initial proposal, then weakening, with no (K) check. Φ°: refutation on forks. The sweep: an optional final strengthening on small instances, the only maximality test ($S_{Q_i} \models S_i$ is $\exists\forall$).

4 What can be proved and what cannot

Theorem 3 (soundness by construction)**.** *If the loop returns a family, it satisfies (C0), (C1), (V1), (V2), (K): it is an acceptable delegation and each contract mentions only its own ports.*

Proof. A family is returned only after the verifier accepts it (lines 7, 11) or after a weakening step from an accepted family (line 23), which checks (C1), (V1), (V2) and inherits (K) from $S_i \models S_i'$. □

The trust base is Z3's unsat answers in quantifier-free linear real arithmetic, Z3's answer on the (K) sentence when no rule list is given, the compiler $[\cdot]_0$ from PolySTL to $\mathcal{L}$, and the verifier's Python code that parses proposals, checks (C1) and rule-list syntax, and builds S_i, $S_i[f_i]$ and the (V1), (V2) formulas. An answer "unknown" counts as a rejection.

Claim 4 (semi-decision at every horizon; claimed in `notes/agentic/formulation.md`, to be proved in work package 1)**.** *If an acceptable delegation exists, one exists in which, for every $i \in N$, the assumption is* `true` *and S_i is $[\![f_i]\!]$ for a causal rule list f_i with rational data. Hence a fair enumeration of tuples of such rule lists (each tuple appears at a finite position), with one quantifier-free check each ($A_P \wedge \bigwedge_i [\![f_i]\!] \wedge \neg G_P$ unsat), halts with an acceptable delegation iff one exists.*

Argument: Lemma 1(b) gives rule lists, $[\![f_i]\!] \models S_i$, (V1) survives shrinking, (V2) is trivial, (K) holds by the rule lists. `complete.tex` states this only for $H = 0$. **Completeness** holds only relative to fairness of the proposals, which a language model does not promise. Interleaving gives it back: the proposer of component i returns the enumerator's next candidate every k-th time it is asked; that this reaches every tuple under routing is part of work package 1. On nested orderings with Φ decided, lines 2–4 give completeness, not the proposers. **Non-existence** is shown only by Φ or Φ° false; a true Φ° proves nothing (Left/Right in `complete.tex`). Decidability in general stays open. **Termination** is by the budget, or when all asked components decline on one failure (line 14), or when weakening finds no positive example in a full cycle (line 21); none of these says anything about existence. Only the enumerator of work package 3, which checks every tuple of a finite candidate set, can say "none in this set" (not "none exists"). **Reproducibility.** Quantifier-free verdicts are deterministic: sat or unsat in quantifier-free linear real arithmetic does not depend on solver state (models do; building formulas in sorted order made the governor output identical under four hash seeds). Language-model runs cannot be reproduced by re-running: the models in question accept no temperature other than the default, and the API has no seed. The reproducible artefact is the log of proposals (raw text, parsed formula, verdict, exact counterexample, versions), a certificate file (graph, P, H, contracts, rule lists), one command `verify.py certificate.json` that re-checks every constraint with no proposer, and `replay.py log.jsonl`, which re-verifies each logged proposal. Language-model results are reported over several runs. **Canonicity.** `SCOPE.md` asks for a procedure "choosing canonically from (G, P) alone". A deterministic enumerator meets this (its output is a function of (G, P) and a fixed order, as with the candidate order of `complete.tex`); a language model does not. This is a scope consequence the author must accept or reject: either the deliverable uses the enumerator and the language model only speeds the search, its output recorded, or `SCOPE.md` is relaxed to "reproducible from the record".

5 Evidence so far

Instance. The speed governor of `docs/example.tex`: Perception reads o, writes e; Controller reads e, writes u; Actuator reads u, writes v; $A_P = \Box_{[0,2]}(0 \le o \le 5)$, $G_P = \Box_{[0,2]}(0 \le v \le 2) \wedge \Box_{[0,1]}(o \le 1 \to \Diamond_{[0,1]} v \le \frac{1}{4}) \wedge \Box_{[0,1]}(o \ge 3 \to \Diamond_{[0,1]} v \ge 1)$. Two hand-written families, all assumptions true, rule lists in brackets. D_{react}: Perception $\Box_{[0,2]}((o \ge 3 \to e \ge 3) \wedge (o \le 1 \to e \le 1))$ $[e(t) := o(t)]$; Controller the same from e to u $[u(t) := e(t)]$; Actuator $\Box_{[0,2]}(0 \le v \le 2 \wedge (u \ge 3 \to v \ge 1) \wedge (u \le 1 \to v \le \frac{1}{4}))$ $[v(t) := 1$ if $u(t) \ge 3$, else $0]$. D_{lazy}: Perception $o(1) \ge 3 \to e(1) \ge 3$; Controller $e(1) \ge 3 \to u(1) \ge 3$; Actuator $\Box_{[0,2]}(0 \le v \le 2) \wedge v(0) \ge 1 \wedge v(1) \le \frac{1}{4} \wedge (u(1) \ge 3 \to v(2) \ge 1)$ $[v(0) := 1, v(1) := 0, v(2) := 1$ if $u(1) \ge 3$, else $0]$. One command, `.venv/bin/python notes/agentic/governor_checks.py`, prints 101 verdicts, all as expected, in 0.26 s (times are machine-dependent):

family (every check passes)	(V1)	(V2), all j	(K) by rule list	(K) by the sentence
D_{react}, $H = 2$	2.9 ms	0.9 ms	0.6 ms (slowest)	6.9 ms (slowest)
D_{lazy}, $H = 2$	0.5 ms	0.8 ms	0.3 ms (slowest)	1.1 ms (slowest)
swept family of `example.tex`, $H = 2$	0.6 ms	0.9 ms	no rule list given	1.5 ms (slowest)
D_{react}, $H = 80$ (324 timed variables)	12.1 ms	trivial (all $A_j = \mathsf{true}$)	15.5 ms (all three)	not run
`complete.tex` route, $H = 2$	Φ 0.03 s; seed 0.02 s; whole procedure 0.5 s			

Maximal is not greatest. S_{Actuator} of D_{lazy} entails the swept one, but $S_{\text{Perception}}$ and $S_{\text{Controller}}$ do not; so D_{lazy} is acceptable and not inside the maximal swept family. Weakening and a repair by projected cell (computed in 2.4 ms) also pass; a foreign port, $A = \mathsf{false}$, $(\mathsf{true}, \mathsf{false})$, input pinning and two bad rule lists are rejected. All proposals are hand-written stand-ins: no language model has been run, and `verify.py`, `replay.py` do not exist yet. Today Z3 5.3.1's virtual-substitution tactic was found to return a wrong, too strong elimination for the Actuator, accepted by a cross-check built on the same tactic; `scripts/delegate.py` now uses the model-based tactic with an independent cross-check, runs in 2 s, and asserts the swept Actuator set against the one above. The sweep's cost is an issue on the three-component, two-input stack (nine minutes, unfinished), not on the governor.

6 Building it with Strands

Version. `strands-agents` 1.61.1 (15 October 2026), Python $\ge$ 3.10, Apache-2.0. `AnthropicModel` calls the Anthropic API directly; `structured_output_model=Contract` returns a Pydantic object, re-asking on validation errors.[1] **Shape.** One `Agent` per component; a `Contract` of SMT-LIB 2 strings, parsed with only the component's ports declared, so (C1) is free; hooks logging every call. Swarm and Agents-as-Tools are rejected: a language model would route. **Strands Labs.** AI Functions (`strands-ai-functions` 0.5.0, Python $\ge$ 3.12) turns a typed Python function into an agent call with post-conditions.[2] It fits (C1) and (K), not (V1), (V2), which need every proposal. **Blockers.** (i) Opus 5.5 and Sonnet 5.5 reject a forced tool choice, which Strands sends when the model ends a turn without calling the output tool; use a `submit_contract` tool and log failures as failed proposals (untested live). (ii) No sampling control: no temperature other than the default, no seed (Opus 5.5 migration guide). (iii) Weekly releases: pin

[1] `https://strandsagents.com/docs/user-guide/sdk/agents/structured-output/`
[2] `https://github.com/strands-labs/ai-functions`

`strands-agents[anthropic]==1.61.1`, `anthropic==0.125.0`. (iv) Bedrock is the silent default without an explicit model.

7 Related work

As far as a search of about 30 arXiv queries shows, nobody uses language-model or agent proposers for assume–guarantee contract decomposition. The closest pieces: Katis et al. check realizability of one component's contract over linear real arithmetic and extract implementations [1, 2], usable as our (K) checker. Golia, Roy and Meel cast synthesis as dependency quantified formulas [3], the logical form of (K) with port locality, and Manthan3 [4] is the learn–verify–repair loop over Booleans that ours follows. Skolemization for quantified linear real arithmetic by Chatterjee et al. [5] is the natural symbolic proposer for one component's (K), and a baseline. Agentic Model Checking [6] is the closest architecture, without a contract algebra, real-valued time or receptiveness. Finkbeiner, Metzger and Moses show that some necessary obligations are about information flow, not behaviour [7]: this is why forwarding stays symbolic. The limits: assume–guarantee synthesis under partial information is undecidable in many settings [8], and counterexample-guided synthesis need not terminate over infinite candidate spaces [9].

8 Work packages

WP1 and WP2 in parallel, then WP3 to WP6 in order; about 30 working days.

- **WP1 Proofs (5 days).** Re-check Lemma 1 (proved here); prove the semi-decision claim, the routing policy (cells stay forbidden while the other sets grow; weakening keeps acceptability), and completeness when interleaved.
- **WP2 Verifier library (6 days).** `verify.py` (all constraints, Lemma 2(b),(c), cells, certificates) and `replay.py`. *Done when* it reproduces the 101 verdicts as a regression suite, identically under four hash seeds.
- **WP3 Enumerator proposer (5 days).** Rule lists by increasing size and coefficient set; first accepted output, or "none in this set". Baseline and canonical. *Done when* it solves the relay, governor ($H = 2$) and fan-in, identically each run.
- **WP4 Language-model proposer in Strands (5 days).** Pinned versions, Anthropic provider, hooks logging every model call. *Done when* relay and governor runs pass within budget and replay reproduces each verdict.
- **WP5 Experiments (6 days).** Governor at $H = 2$, 10; the three-component, two-input stack the sweep could not finish (to be written down first: it is not in this directory); Left/Right forks for Φ°. Measure rounds to acceptance, check time per round, model calls, cost; ten language-model runs per instance.
- **WP6 Document and reproduction (3 days).** Method and results, every number printed by one command.

Stop criteria: the agentic route is also not fruitful if (a) on the governor at $H = 10$ and the three-component stack neither proposer is accepted within 50 rounds in most runs; (b) checks per round take longer than the sweep; or (c) on forks neither proposer finds a family and Φ° refutes nothing. If only the language model loses to the enumerator, drop WP4.

9 Risks

- *Invented ports* fail (C1), since the parser declares only the component's ports. *Vacuous or input-pinning proposals.* $(\mathsf{true}, \mathsf{false})$ has empty S_i and fails (R), hence (K). Pinning an input $(\Box_{[0,2]}(u = 0 \wedge v = 0))$ fails (R) at every other input history. $A_i = \mathsf{false}$ fails (V2) if A_P is satisfiable over environment variables only: every S_j already passed (K), hence (R), so $A_P \wedge \bigwedge_j S_j$ is satisfiable.
- *The proposer sees the global query:* allowed, as the problem gives it as an input; only the output must be port-local ((C1)). *Mutual blame loops:* cut by deterministic routing, the no-repeat record and the budget. *Cost:* about \$0.11 per Opus 5.5 call (an estimate), under \$2 for the relay; logged per call. *API churn:* pinned versions, a lock file, replay without a model.

References

[1] A. Katis et al. *Synthesis from Assume-Guarantee Contracts using Skolemized Proofs of Realizability*. 2016. arXiv: 1610.05867.

[2] A. Katis et al. *Validity-Guided Synthesis of Reactive Systems from Assume-Guarantee Contracts*. 2017. arXiv: 1709.04986.

[3] P. Golia et al. *Program Synthesis as Dependency Quantified Formula Modulo Theory*. 2021. arXiv: 2105.09221.

[4] P. Golia et al. *Synthesis with Explicit Dependencies*. 2023. arXiv: 2301.10556.

[5] K. Chatterjee et al. *Quantified Linear and Polynomial Arithmetic Satisfiability via Template-based Skolemization*. 2024. arXiv: 2412.16226.

[6] Y. Sun et al. *Agentic Model Checking*. 2026. arXiv: 2605.21434.

[7] B. Finkbeiner et al. *Information Flow Guided Synthesis (Full Version)*. 2022. arXiv: 2205.12085.

[8] R. Bloem et al. *Assume-Guarantee Synthesis for Concurrent Reactive Programs with Partial Information*. 2014. arXiv: 1411.4604.

[9] S. Jha and S. A. Seshia. *Are There Good Mistakes? A Theoretical Analysis of CEGIS*. 2014. arXiv: 1407.5397.